Coding Assignment 8: Binary Search and Its Variants

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 15, 2026

Concepts That Click

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
This assignment autograds Week 8's institute practical on **binary search and its variants**. The contract is given to you in `search.h` — *do not modify it*. Copy `starter_search.c` to `search.c` and fill in the five functions. The setting is the running calculator: its symbol table is a sorted array of identifier ids, and this module is how the calculator searches it — the ordinary lookup, the first/last-declaration boundaries that count duplicate declarations, and the integer square root used to size a bucket.

8.1 Why this problem

Week 8's whole argument is that *order* turns an $O(n)$ scan into an $O(\log n)$ probe — and that binary search is only correct on data that actually is sorted (the deck's failure trace on unsorted input is the warning). The lecture builds the ordinary search, the `lowerBound`/`upperBound` boundary finders, and one application where the value searched for is not stored in any array at all: the integer square root, found by binary search on a monotone predicate. This assignment is that material as running code. The one extension beyond the deck's own listings is `countOccurrences`, which the lecture only names as the identity `upperBound − lowerBound` — you implement it and must keep it $O(\log n)$, not a re-scan.

8.2 Your task

Implement the five functions declared in `search.h`, all operating on a sorted (nondecreasing) table `A[0..n-1]`, `n >= 0`:

```
int binarySearch(const int *A, int n, int key);      /* some index, or -1 */
int lowerBound(const int *A, int n, int key);        /* first A[i] >= key */
int upperBound(const int *A, int n, int key);        /* first A[i] >  key */
int countOccurrences(const int *A, int n, int key);  /* count of A[i]==key */
int isqrtFloor(int n);                               /* floor(sqrt(n)), n>=0 */
```

The three search routines. `binarySearch` is the deck's iterative search: keep an inclusive range `[lo, hi]`, probe the middle, keep the half that can still contain `key`, and return `-1` when the range empties. Compute the midpoint as `lo + (hi - lo) / 2`, never `(lo + hi) / 2` — the lecture's overflow caution. The result is *some* matching index; any valid one is accepted. `lowerBound` and `upperBound` use a *half-open* range `[lo, hi)` with `hi = n`: to find the *first* qualifying index,

keep `mid` as a candidate (`hi = mid`) instead of discarding it, and return `lo` when the range closes. `countOccurrences` must be exactly `upperBound − lowerBound`: calling your own two bounds is the intended route, and it must not scan the table.

The application. `isqrtFloor` returns $\lfloor\sqrt{n}\rfloor$ — the largest integer $k \geq 0$ with $k^2 \leq n$ — using *integer arithmetic only* (no `<math.h>`, no floating point, no library `sqrt`). Search the answer space $0..n$ on the monotone predicate $P(k) = (k^2 \leq n)$: when P(mid) is true, record `mid` and push right; otherwise push left. *Widen before squaring* — a signed 64-bit type such as `(long long)mid * mid <= n`. On Windows `long` is only 32 bits, so the lecture's `(long)` cast overflows `mid * mid` for large n; the portable form uses `long long`.

8.3 Constraints

- The table is sorted and read-only; none of the search functions may modify `A`. The precondition is a *contract* — binary search on unsorted data is silently wrong, which is why the tests only ever pass sorted tables.
- `n == 0` is the empty table: `binarySearch` returns -1; `lowerBound`, `upperBound`, and `countOccurrences` return 0 (and `A` may be `NULL` then).
- `isqrtFloor` requires `n >= 0` and must not use floating point or a library square root.
- Complexity: all five functions are $O(\log n)$ worst case — a single halving loop, or (for `countOccurrences`) two of them — with $O(1)$ auxiliary space. The tests include a 100,000-element table and `isqrtFloor(INT_MAX)`, so a linear scan or a 32-bit squaring overflow will not pass.
- Your code must compile with `gcc -std=c11 -Wall -Wextra -Werror` without errors or warnings, and must have no global or static mutable state.

8.4 Worked examples (these are the visible tests)

All on the sorted table `[4, 9, 15, 15, 22, 31, 47, 58]` (`n = 8`):

Example 1 — ordinary search. `binarySearch` for 22 returns 4; for 4 returns 0; for 58 returns 7; for 16 (absent) returns -1; for 100 (above all) returns -1; and on the empty table returns -1.

Example 2 — the two bounds and the count. `lowerBound(15) = 2` and `upperBound(15) = 4`, so `countOccurrences(15) = 2`. A value that is present once gives a one-wide band: `lowerBound(16) = 4`, `upperBound(16) = 4`, count 0 (it is absent). Below and above the whole table: `lowerBound(1) = 0`, `upperBound(1) = 0`; `lowerBound(99) = 8`, `upperBound(99) = 8`. At the ends: `lowerBound(4) = 0`, `upperBound(4) = 1`; `lowerBound(58) = 7`, `upperBound(58) = 8`.

Example 3 — integer square root. $\lfloor\sqrt{0}\rfloor = 0$, $\lfloor\sqrt{1}\rfloor = 1$, $\lfloor\sqrt{2}\rfloor = 1$, $\lfloor\sqrt{3}\rfloor = 1$, $\lfloor\sqrt{4}\rfloor = 2$; $\lfloor\sqrt{15}\rfloor = 3$ (the last k with $k^2 \leq 15$ is 3, since $9 \leq 15 < 16$); $\lfloor\sqrt{16}\rfloor = 4$ and $\lfloor\sqrt{17}\rfloor = 4$; $\lfloor\sqrt{26}\rfloor = 5$; $\lfloor\sqrt{48}\rfloor = 6$; $\lfloor\sqrt{63}\rfloor = 7$.

8.5 What to submit

1. `search.c` — your completed implementation (edit `starter_search.c`).

Concepts That Click

2. `search.h` — unchanged, included so the submission compiles standalone.

8.6 Getting started

1. Copy `starter_search.c` to `search.c` and fill in the five function bodies.
2. Sanity-check locally:

   ```
   gcc -std=c11 -Wall -Wextra -Werror -o test_visible test_visible.c search.c
   ./test_visible
   ```

   All three worked examples should pass.
3. Submit `search.c` and `search.h`. The autograder compiles your `search.c` against a *hidden* test suite and reports pass/fail. The hidden suite covers: `binarySearch` on empty/single/two/all-duplicate tables; the target at index 0 and at the last index; targets absent below, between, and above the range; negative values and the `INT_MIN`/`INT_MAX` endpoints; a 100,000-element table (the index-arithmetic/overflow class); and seeded random tables where the returned index must be a genuine occurrence. `lowerBound`/`upperBound`/`countOccurrences` on empty/single/all-duplicate/no-duplicate tables, keys below all / above all / between values, negative and `INT` extremes, with seeds checked against linear oracles and the identity `count == upper - lower`. `isqrtFloor` over a full 0..5000 sweep, every perfect square k^2 and each of $k^2 - 1$ and $k^2 + k$ up to $k = 46340$, `INT_MAX` and its neighbours (the 32-bit squaring-overflow class), and seeded random arguments checked against a brute-force oracle.

Concepts That Click